

RESTAURANT POS & BILLING SYSTEM

DECLARATION

I hereby declare that the project titled **“Restaurant POS & Billing System”** is an original work developed by me. This project demonstrates a real-world implementation of restaurant order management, billing computation, inventory tracking, and daily sales analytics using Python.

TABLE OF CONTENTS

1. Executive Summary
 2. Introduction
 3. Problem Statement
 4. Proposed Solution
 5. System Architecture
 6. Technology Stack
 7. Functional Modules
 8. Data Structure Design
 9. Billing Engine Logic
 10. Order Workflow
 11. Error Handling & Validation
 12. Advanced Features
 13. Testing Strategy
 14. Performance & Scalability
 15. Future Enhancements
 16. Conclusion
 17. GitHub Repository
-

1. EXECUTIVE SUMMARY

The Restaurant POS & Billing System is a console-based restaurant management simulation built using Python.

The system replicates real-world restaurant operations such as:

- Menu management
- Order taking
- Bill calculation with GST & service charge
- Automatic inventory updates
- Daily sales tracking

The project demonstrates structured programming, modular system design, and real-time billing computation.

2. INTRODUCTION

Modern restaurants require:

- Fast billing systems
- Accurate tax calculations
- Inventory control
- Sales monitoring
- Order validation

This project simulates these core operations using Python in a command-line interface environment.

The application reflects practical POS (Point of Sale) system logic used in restaurants.

3. PROBLEM STATEMENT

Manual billing systems can cause:

- Calculation errors
- Inventory mismanagement
- Poor sales tracking
- Order confusion
- Lack of daily revenue insights

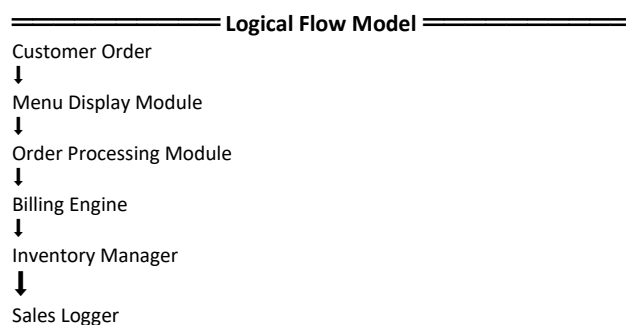
The challenge was to build a structured restaurant POS system that automates billing, updates inventory automatically, and logs daily sales efficiently.

4. PROPOSED SOLUTION

A modular Restaurant POS system that:

- Uses nested dictionaries for menu storage
 - Validates stock before confirming orders
 - Automatically calculates GST & service charge
 - Updates inventory after each confirmed order
 - Maintains daily sales summary
 - Handles invalid inputs gracefully
-

5. SYSTEM ARCHITECTURE



Architecture Layers

- ❖ **Presentation Layer**
Handles CLI interaction and menu navigation.
 - ❖ **Business Logic Layer**
Processes item selection, quantity validation, and bill calculation.
 - ❖ **Inventory Layer**
Reduces stock automatically after order confirmation.
 - ❖ **Sales Tracking Layer**
Stores order summaries and calculates total revenue.
-

6. TECHNOLOGY STACK

===== CORE STACK =====

- **Backend:** Python, Django (MTV Architecture)
- **Frontend:** HTML5, CSS3, JavaScript (Bootstrap 5, BI Icons)
- **Database:** MySQL / PostgreSQL / SQLite
- **Reporting:** ReportLab (PDF Engine), OpenPyxl (Excel Generation)
- **Interface:** Command Line Interface (CLI)
- **Data Handling:** In-memory dictionaries & lists

===== PYTHON CONCEPTS USED =====

- Nested Dictionaries
- Lists
- Functions
- Loops
- Conditional Statements
- Exception Handling
- Input Validation
- Modular Programming

===== DEVELOPMENT TOOLS =====

- VS Code
 - Git
 - GitHub
-

7. FUNCTIONAL MODULES

MENU MANAGEMENT MODULE

- Add new items
- Update price
- Update stock
- View menu

ORDER MODULE

- Select items
- Enter quantity
- Validate stock availability
- Confirm order

BILLING MODULE

- Subtotal calculation
- GST percentage addition
- Service charge calculation
- Final amount display

INVENTORY MODULE

- Auto stock reduction
- Prevents ordering out-of-stock items

SALES TRACKING MODULE

- Daily revenue calculation
 - Most sold items tracking
 - Order logging
-

8. DATA STRUCTURE DESIGN

Menu Data Format

```
{  
  "Burger": {"price": 120, "stock": 50},  
  "Pizza": {"price": 250, "stock": 30},  
  "Coffee": {"price": 80, "stock": 100}  
}
```

Why Nested Dictionaries?

- Fast item lookup
- Organized structure
- Easy stock update
- Scalable for large menus

Daily Sales Format

```
{  
"orders": [order_summary_list],  
"total_revenue": Float  
}
```

9. BILLING ENGINE LOGIC

Bill Formula:

Subtotal = Sum (item price × quantity)

GST = Subtotal × GST%

Service Charge = Subtotal × Service%

Final Amount = Subtotal + GST + Service Charge

The billing engine ensures:

- Accurate tax calculation
 - Automatic rounding
 - Clear bill breakdown display
-

10. ORDER WORKFLOW

Step 1 ➤ Launch Application
Step 2 ➤ Display Main Menu
Step 3 ➤ Select Order Option
Step 4 ➤ Choose Items
Step 5 ➤ Enter Quantity
Step 6 ➤ Validate Stock
Step 7 ➤ Confirm Order
Step 8 ➤ Generate Bill
Step 9 ➤ Update Inventory
Step 10 ➤ Log Sale

11. ERROR HANDLING & VALIDATION

- Handles invalid item selection
 - Prevents ordering more than available stock
 - Prevents negative quantity input
 - Handles non-numeric entries
 - Validates menu option selection
 - Ensures system stability
-

12. ADVANCED FEATURES

- Coupon code & discount engine
- Role-based access (Manager / Cashier)
- Export bill as text/PDF
- Menu category system (Drinks, Main Course, Dessert)
- Most-sold item analytics dashboard

These features improve scalability and professional value.

13. TESTING STRATEGY

- Unit testing of each module
 - Stock validation testing
 - Tax calculation accuracy testing
 - Boundary testing (zero stock)
 - Invalid input testing
 - Multiple order testing
-

14. PERFORMANCE & SCALABILITY

- Fast bill generation
- Efficient stock updates
- Modular architecture
- Easy feature extension
- Lightweight execution

The system can be upgraded into GUI or web-based POS software.

15. FUTURE ENHANCEMENTS

- GUI version using Tkinter
 - Database integration (MySQL / SQLite)
 - Web-based version using Django/Flask
 - Cloud-based deployment
 - Barcode scanning integration
 - Payment gateway simulation
-

16. CONCLUSION

The Restaurant POS & Billing System demonstrates:

- Strong Python fundamentals
- Real-world billing system simulation
- Accurate tax computation
- Inventory automation logic
- Sales analytics tracking
- Modular and scalable architecture

17. GitHub Repository

Click the link below:

[Restaurant POS Billing System](#)
